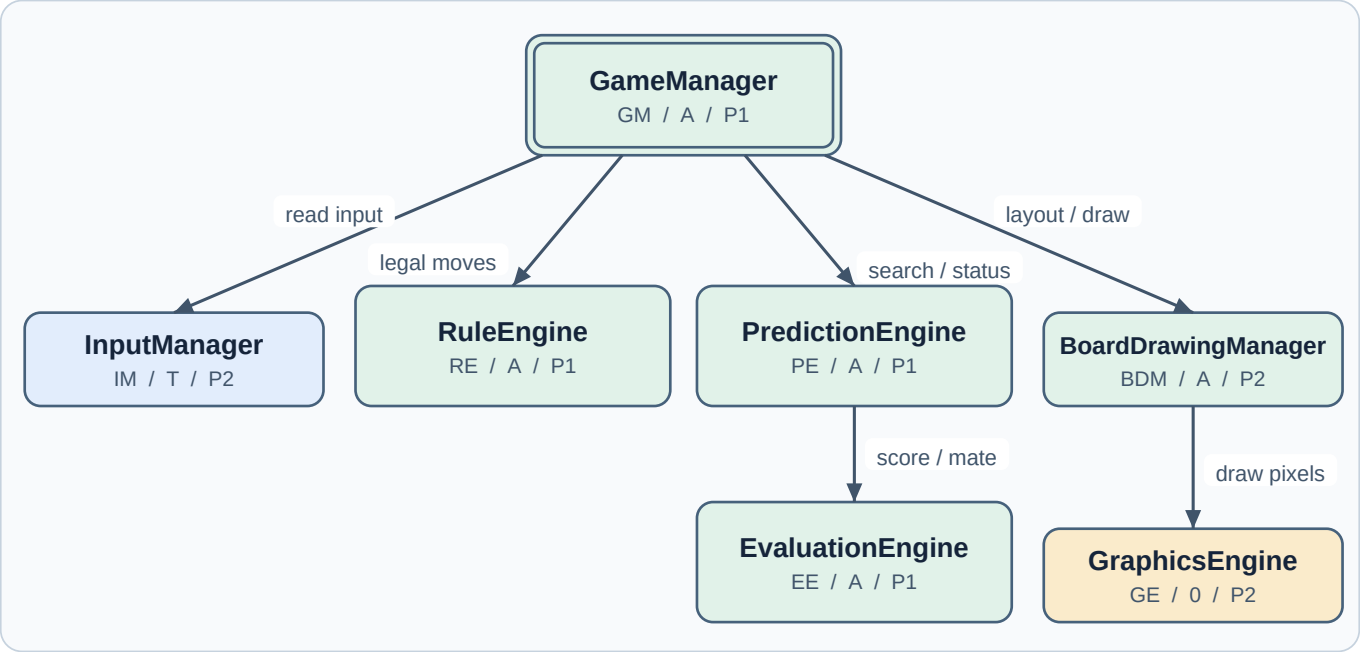
MateMate architecture proposal

Unearth Your Solutions / Queen&King Inc. / SE4 subsystem establishment

MateMate is a desktop chess application for a human playing the computer. The player moves pieces by drag and drop. The computer explores legal positions, selects a reply and shows it on the same board.

Classwork basis. The architecture uses the group's seven subsystem names, A/T/0 classifications, all 20 service assignments and the allowed-to-use table from 3 November 2026. The walkthroughs add interface and state-handling details to those decisions.

Subsystem structure



Key. Green A = chess logic. Blue T = technical input. Amber 0 = generic pixel graphics (the group's 0 classification). Double border = committed-state owner. Large P1 = review-critical. Standard P2 = supporting. Arrows point from caller to allowed provider.

How the boundaries work

GameManager coordinates input, rules, prediction and drawing. It owns the live board, move history and game-level input history. RuleEngine computes legal transitions. PredictionEngine chooses a move by evaluating future positions through EvaluationEngine.

InputManager reports raw events. GameManager uses the layout returned by BoardDrawingManager to map pixels to squares. BoardDrawingManager draws the chessboard and delegates pixel operations to GraphicsEngine.

Design assumptions. One offline desktop process, one in-memory match and a bounded background search. The example starts with the human as White. The same rules and engines handle both colors.

Subsystems and services

The names, blood types and allowed-to-use entries follow the group’s classwork.

Subsystem	Responsibility	Knowledge	May use
IM · T InputManager	Captures mouse and keyboard input.	Button state, raw pointer position and click/drop samples.	None
BDM · A Chess. BoardDrawingManager	Draws the board and chess pieces.	Piece sizes, board geometry, orientation and rendering.	GE
GE · 0 GraphicsEngine	Draws primitives at pixel positions on a canvas.	Drawing coordinates and canvas operations.	None
EE · A ChessCore. EvaluationEngine	Scores positions and identifies checkmate from supplied facts.	Positional scores, piece weights and checkmate criterion.	None
PE · A ChessCore. PredictionEngine	Searches future positions and selects the best explored move.	Search frontier, scores and best completed result.	EE
RE · A ChessCore. RuleEngine	Checks legal movement and computes resulting positions.	Move rules, attack checks and position transitions.	None
GM · A ChessCore. GameManager	Coordinates the game, turns, input and drawing.	Current board, side to move, move history and input history.	PE, RE, BDM, IM

Service allocation

Provider	Services from the classwork
IM	**1** Latest click in screen pixels.
BDM	**3** Draw a white pawn on c4. **6** Draw a position. **12** Report displayed queen size. **18** Draw an empty board.
GE	**5** Draw a black dot at pixel position (898, 266).
EE	**7** Evaluate a position for White. **8** Determine whether White is checkmated. **10** Report rook weight.
PE	**2** Find the optimal move within the configured search budget.
RE	**4** Find legal destinations for the rook on a5.
GM	**Input** · **13** Latest click as a square. **16** Latest drop in screen pixels. **Board** · **9** Side to move. **11** White castling eligibility. **14** Occupant of b2. **15** Whether Black has castled. **History** · **17** Board four moves ago. **19** Black’s latest move. **20** Current match age.

The group’s table permits exactly six direct subsystem dependencies. Raw inputs, positions and rule facts pass as values across those interfaces.

Walkthrough 1 · Make move

The human plays White and drops the e2 pawn at screen position (824, 535).

Resolve the drag

Assume a White-oriented 640 × 640 board with origin (500, 175) and 80-pixel squares. GM reads the press and drop from IM and checks the layout captured from BDM. The drop has column floor((824 - 500) / 80) = 4 and row floor((535 - 175) / 80) = 4, both zero-based. The file is e and the rank is 8 - 4 = 4. The press selected e2, so GM submits **e2-e4** to RE.

Step	Call and data	Result
1	GM reads raw IM events, records the latest drop in pixels and maps the press and drop to squares.	GM creates a square-based move intent for the current match and revision. An out-of-bounds drop or changed layout cancels the drag.
2	GM asks RE to validate the intent and compute the new immutable position.	GM commits once and asks BDM to draw the new snapshot. BDM draws through GE.
3	GM starts a PE search on a worker. When PE returns a request for a position’s legal successors or rule facts, GM obtains them from RE and passes the results back to PE as data.	PE runs bounded minimax with alpha-beta pruning. PE calls EE for scoring and checkmate. White maximizes the score and Black minimizes it.
4	PE returns its candidate. GM checks the match ID, revision and search token, then validates the candidate with RE and commits it.	GM sends the snapshot to BDM, which draws the reply through GE. If PE selects e7-e5, the pawn appears on e5.

Interface and failure behavior

Rule facts. RE supplies whose turn it is, whether that king is in check and whether any legal move exists. PE passes those facts to EE for service 8. EE identifies checkmate when the side to move is in check and has zero legal moves. This distinguishes it from stalemate. GM can obtain match status through PE before starting a search.

An illegal move leaves board state, move history and match revision intact. At timeout GM uses PE’s best completed result or a prevalidated legal fallback. It discards late or duplicate results and skips search if the game has ended.

Walkthrough 2 · Castle

GameManager owns castling state. RuleEngine checks and computes its transitions.

Side	King move	Rook move	Required right
White kingside	e1 to g1	h1 to f1	WK
White queenside	e1 to c1	a1 to d1	WQ
Black kingside	e8 to g8	h8 to f8	BK
Black queenside	e8 to c8	a8 to d8	BQ

The starting position grants all four rights. RE clears both rights for a color when its king moves. It clears the matching right when an original rook moves off its home square or is captured there. Returning a king or rook to its starting square or putting another rook there cannot restore a lost right.

RE checks the side to move, the king and eligible rook on their home squares, an empty path and safety on the king's start, transit and destination squares. A legal castle moves both pieces in one transition. GM records one accepted move and uses the new state to answer whether a color has castled. It reports retained castling rights separately from whether castling is legal in the current position.

Review fixtures. Test all four legal castles, then try castling after the king or rook moved and returned, after the original rook was replaced, with a blocked path and through check. Rejection preserves the board, castling rights, move history and revision.

State and search decisions

GM keeps the accepted move history separate from the immutable position snapshot within the same subsystem. GM uses RE transitions for both live and hypothetical moves. A worker uses its own branch-local repetition history. The repetition key includes board, side to move, castling rights and a legally available en passant capture.

GM serializes updates to the live match. The worker only returns candidates against its starting revision. This keeps speculative search separate from actual moves and lets GM reject results from an earlier turn or match.

Sources

MateMate group classwork, *Services*, 3 November 2026 (SWA_Services_261103.docx).

Martin Hasitschka, *Architecture Documentation*, V704, PDF pp. 5, 9-10 and 72.

Martin Hasitschka, *Architecture Development*, V702, PDF pp. 27-36.

FIDE Laws of Chess, arts. 3.8-3.9 and 9.2-9.6.